

```
public ExampleFinalizer() { Console.WriteLine("In ctor");  
}  
~ExampleFinalizer() { Console.WriteLine("In dtor"); }  
public static void Test()  
{  
    for (int i = 0; i < 50000; i++)  
    {  
        new ExampleFinalizer();  
    }  
}  
}
```

That being said, how do we provide deterministic finalisation? Resources typically provide some operation `close()` that "frees" the resource. Without additional language support, all we can do is make sure we call the method when we are done. Try-finally allows us to do this in a safe way, but it can become incredibly unwieldy.

```
public static class ManualResourceTest  
{  
    public static void Test()  
    {  
        StreamReader sw = null;  
        try  
        {  
            try  
            {  
                sw = new StreamReader(@"C:\Users\posera\myfile.  
txt");  
            }  
            catch (FileNotFoundException ex)  
            {  
                Console.WriteLine("File not found");  
                return;  
            }  
            try  
            {  
                Console.WriteLine(sw.ReadToEnd());  
            }  
            catch (IOException ex)
```